

1. Introduction

The hospitality industry relies heavily on understanding customer behavior to optimize resource allocation. The goal of this project is to develop a machine learning system that performs Customer Segmentation to identify distinct types of hotel guests and subsequently predicts which guest segment is most likely to visit during specific seasons.

Unlike traditional booking prediction models that simply forecast cancellation rates, this project employs a Hybrid Learning Approach. First, we use Unsupervised Learning (Clustering) to group guests based on complex features such as spending habits (adr), special requests, and stay duration. Second, we use Supervised Learning to train an Artificial Intelligence model to predict these guest clusters based on temporal features (Date and Season). This allows the hotel management to anticipate specific "Guest Profiles" for any given date, enabling better preparation of rooms, meals, and staffing.

2. Dataset Description

The shape of the dataset is (119390, 32). Meaning there are 32 features and 119390 data points. This is a classification problem. Because we are classifying our clusters in a set of labels. There were both quantitative data like adr, lead_time etc and categorical data like "City Hotel", "Resort Hotel", different months. I encoded the categorical to a numerical form using One hot encoding technique as there were numerous number of ranged values. The data set had empty, null, "Undefined" values which was handled by data imputing. In this case, those spaces were filled up with any number between the top 3 appeared values. While inspecting correlation matrix, it tells that, the 'reddish' part has a correlation between row feature and column feature. We should exclude those characteristics, as it could confuse the model that which number it should take

3. Data PreProcessing

'hotel', 'meal', 'country', 'market_segment', 'distribution_channel', 'reserved_room_type', 'customer_type' they had categorical values and the rest had quantitative values. I have included the features that are used in the notebook, there were some other less necessary features which was excluded from my attention. Handled the non numerical values with one hot encoding. After that, the 'already numerical' numbers were standardized using StandardScaler.

4. Data Splitting

Data was splitted into two parts. 80% was used for training models and 20% for testing. For the neural model, among the test data, I have taken 20% of it for later validation. To check validation losses.

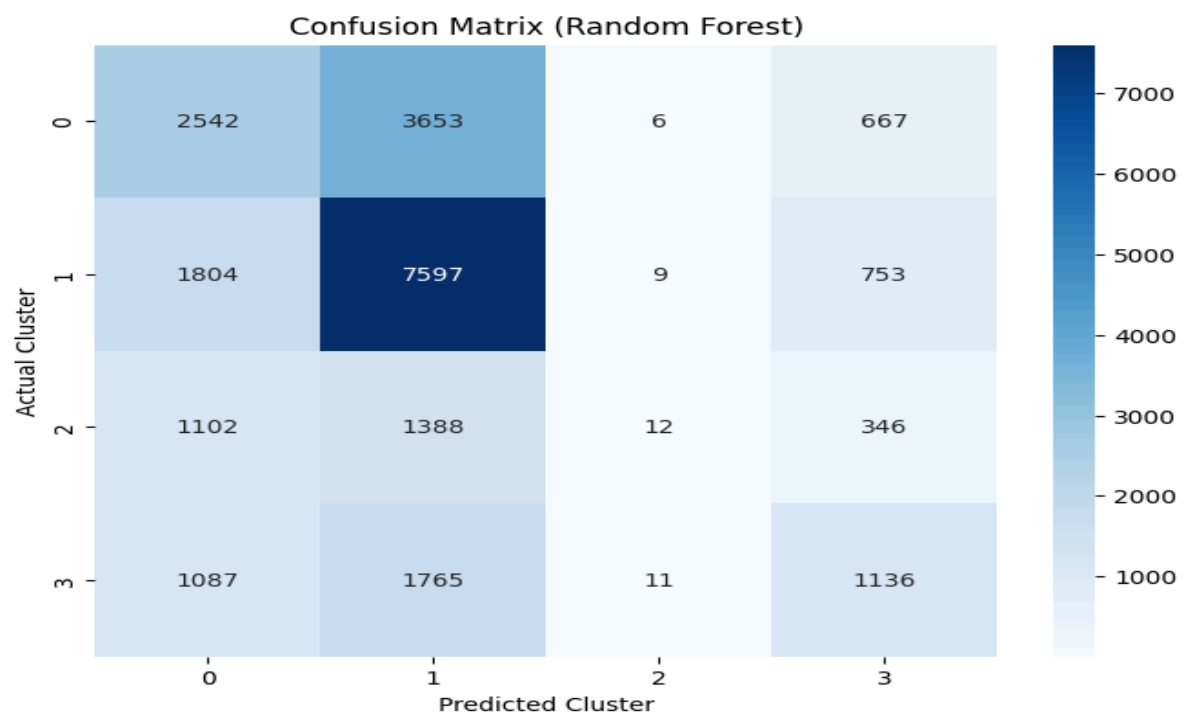
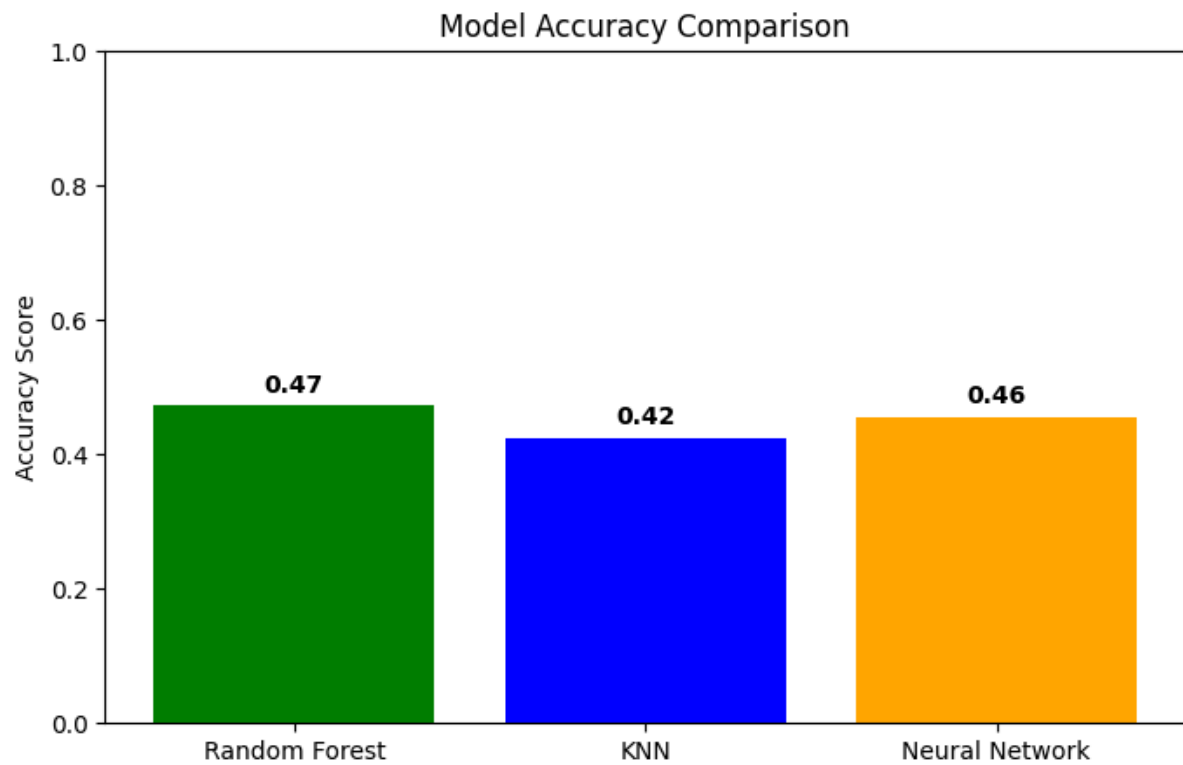
5. Model training and testing(Supervised)

Initially, the problem was seen as an unsupervised problem. Then we clustered into 4 kmeans, which means now it has 4 labels. After that, I used Random forest and KNN algorithms to train model. Finally, fed the neural network the preprocessed data

6. Model selection/Comparison

Precision, recall, F1 score comparison

Random Forest Report		precision	recall	f1-score	support
	0	0.39	0.37	0.38	6868
	1	0.53	0.75	0.62	10163
	2	0.32	0.00	0.01	2848
	3	0.39	0.28	0.33	3999
accuracy				0.47	23878
macro avg		0.41	0.35	0.33	23878
weighted avg		0.44	0.47	0.43	23878
KNN Report		precision	recall	f1-score	support
	0	0.35	0.38	0.36	6868
	1	0.50	0.64	0.56	10163
	2	0.17	0.03	0.05	2848
	3	0.32	0.23	0.27	3999
accuracy				0.42	23878
macro avg		0.34	0.32	0.31	23878
weighted avg		0.39	0.42	0.40	23878
Neural Network Report		precision	recall	f1-score	support
	0	0.38	0.34	0.36	6868
	1	0.49	0.80	0.61	10163
	2	0.00	0.00	0.00	2848
	3	0.36	0.09	0.15	3999
accuracy				0.46	23878
macro avg		0.31	0.31	0.28	23878
weighted avg		0.38	0.46	0.39	23878



Random Forest gives us the best accuracy score,if we look into these plots.

7. Conclusion

Accuracy of KNN, Neural Networks and Random forest is 0.42, 0.46 and 0.47 respectively. Which indicates that Random forest successfully predicted 47% of the client base (clusters) out of the test data. Although, the parameters were randomly assigned but due to hardware limitations, couldn't do fine tuning to achieve a better result.